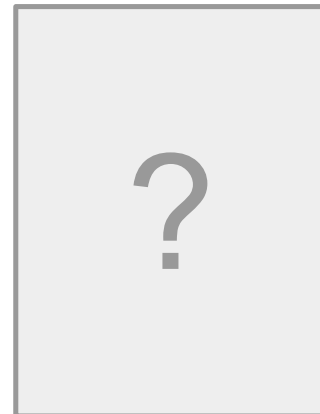
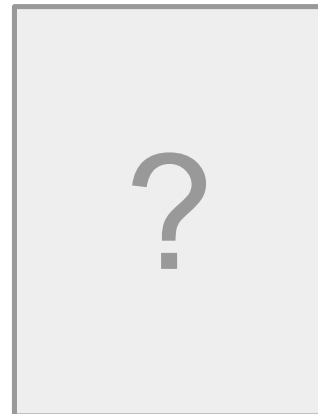
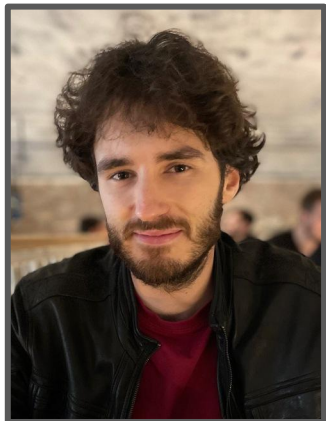




Natural Language Processing

Lecture 3: Transition to Modern Machine Learning.

Course Instructors



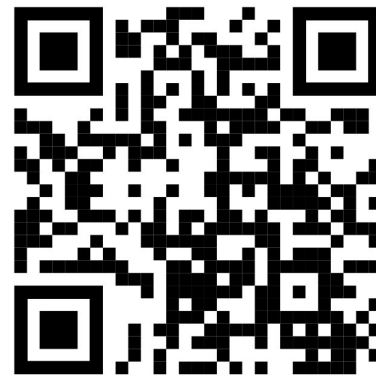
Maksym Shamrai

PhD Student @ Institute of Mathematics of NASU

CV & RL Lecturer @ KAU

AI Research Scientist @ MacPaw

LinkedIn:



Agenda

1. Word meaning
2. Word2Vec
3. GloVe
4. Language Modeling
5. RNNs

Acknowledgement

Some slides are adapted from [CS224N: Natural Language Processing with Deep Learning](#). We express our gratitude to Stanford University.

How do we represent the meaning of a word?

Definition: meaning (Webster dictionary)

- the idea that is represented by a word, phrase, etc.
- the idea that a person wants to express by using words, signs, etc.
- the idea that is expressed in a work of writing, art, etc.

signifier (symbol) $\Leftrightarrow$ signified (idea or thing)

tree $\Leftrightarrow$ {  ,  ,  ,  ,  ,
 , ... }

How do we have usable meaning in a computer?

Previously commonest NLP solution: Use, e.g., **WordNet**, a thesaurus containing lists of synonym sets and hypernyms (“is a” relationships)

e.g., synonym sets containing “good”:

```
from nltk.corpus import wordnet as wn
poses = { 'n': 'noun', 'v': 'verb', 's': 'adj (s)', 'a': 'adj', 'r': 'adv' }
for synset in wn.synsets("good"):
    print("{}: {}".format(poses[synset.pos()],
        ", ".join([l.name() for l in synset.lemmas()])))
```

```
noun: good
noun: good, goodness
noun: good, goodness
noun: commodity, trade_good, good
adj: good
adj (sat): full, good
adj: good
adj (sat): estimable, good, honorable, respectable
adj (sat): beneficial, good
adj (sat): good
adj (sat): good, just, upright
...
adverb: well, good
adverb: thoroughly, soundly, good
```

e.g., hypernyms of “panda”:

```
from nltk.corpus import wordnet as wn
panda = wn.synset("panda.n.01")
hyper = lambda s: s.hypernyms()
list(panda.closure(hyper))
```

```
[Synset('procyonid.n.01'),
Synset('carnivore.n.01'),
Synset('placental.n.01'),
Synset('mammal.n.01'),
Synset('vertebrate.n.01'),
Synset('chordate.n.01'),
Synset('animal.n.01'),
Synset('organism.n.01'),
Synset('living_thing.n.01'),
Synset('whole.n.02'),
Synset('object.n.01'),
Synset('physical_entity.n.01'),
Synset('entity.n.01')]
```

Q: Problems with resources like WordNet?

Problems with resources like WordNet:

1. A useful resource but **missing nuance**:
 - a. e.g., “proficient” is listed as a synonym for “good”
This is only correct in some contexts
2. **Missing new meanings** of words:
 - a. e.g., wicked, badass, nifty, wizard, genius, ninja, bombest
 - b. Impossible to keep up-to-date!
3. **Subjective**
4. Requires **human labor** to create and adapt
5. **Can't be used to accurately compute word similarity**

Representing words as discrete symbols

Q: One-hot encoding?

Representing words as discrete symbols

One-hot encoding:

a **localist** representation

motel = [0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0]

hotel = [0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0]

Vector dimension = number of words in vocabulary (e.g., 500,000+)

Q: What's wrong?

Representing words as discrete symbols

Example: in web search, if a user searches for “Seattle motel”, we would like to match documents containing “Seattle hotel”

But:

motel = [0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0]

hotel = [0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0]

These two vectors are **orthogonal**

There is no natural notion of **similarity** for one-hot vectors!

Solution: Learn to encode similarity in the vectors themselves

Representing words by their context

- *Distributional semantics*: A word's meaning is given by the words that frequently appear close-by
 - “You shall know a word by the company it keeps” (John Rupert Firth)
- When a word w appears in a text, its **context** is the set of words that appear nearby (within a fixed-size window).
- We use the many **contexts** of w to build up a representation of w

...government debt problems turning into **banking** crises as happened in 2009...
...saying that Europe needs unified **banking** regulation to replace the hodgepodge...
...India has just given its **banking** system a shot in the arm...

These **context words** will represent **banking**

Word vectors

Let's build a dense vector for each word, chosen so that it is similar to vectors of words that appear in similar contexts, measuring similarity as the vector dot (scalar) product

banking =

$$\begin{pmatrix} 0.286 \\ 0.792 \\ -0.177 \\ -0.107 \\ 0.109 \\ -0.542 \\ 0.349 \\ 0.271 \end{pmatrix}$$

Note: word vectors are also called (word) **embeddings** or (neural) word representations

Word2Vec: overview

Word2vec is a framework for learning word vectors (Mikolov et al. 2013)

Idea:

- We have a large corpus (“body”) of text: a long list of words
- Every word in a fixed vocabulary is represented by a **vector**
- Go through each position t in the text, which has a center word c and context (“outside”) words o
- Use the **similarity of the word vectors** for c and o to **calculate the probability** of o given c (or vice versa)
- **Keep adjusting the word vectors** to maximize this probability

Word2Vec: overview

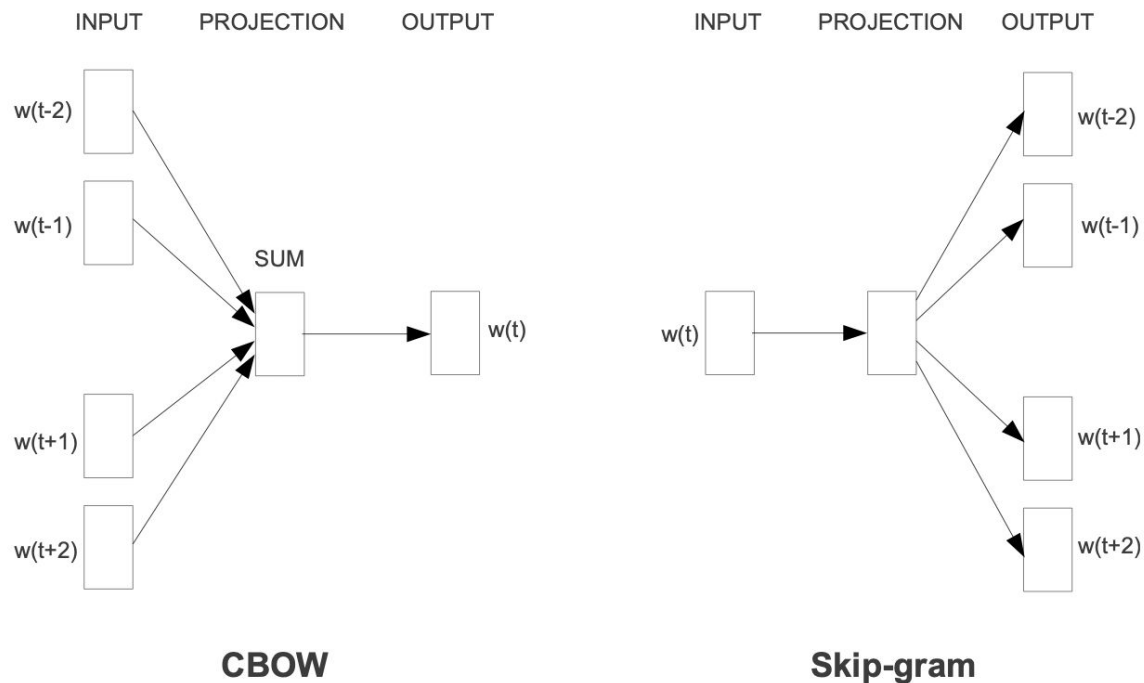
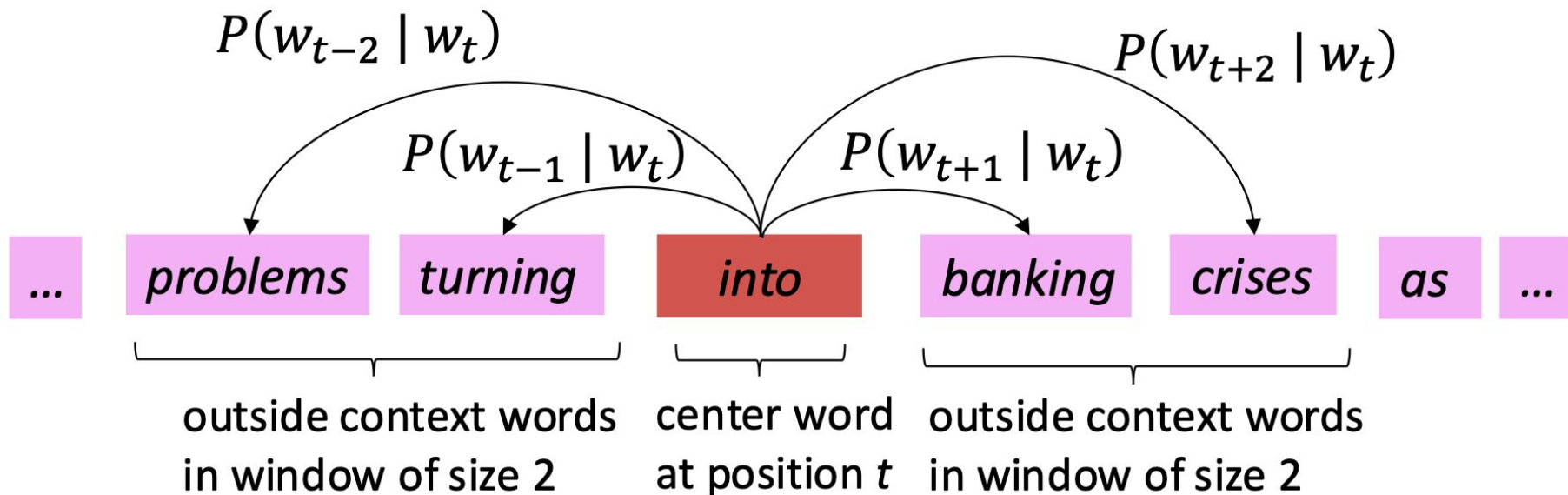
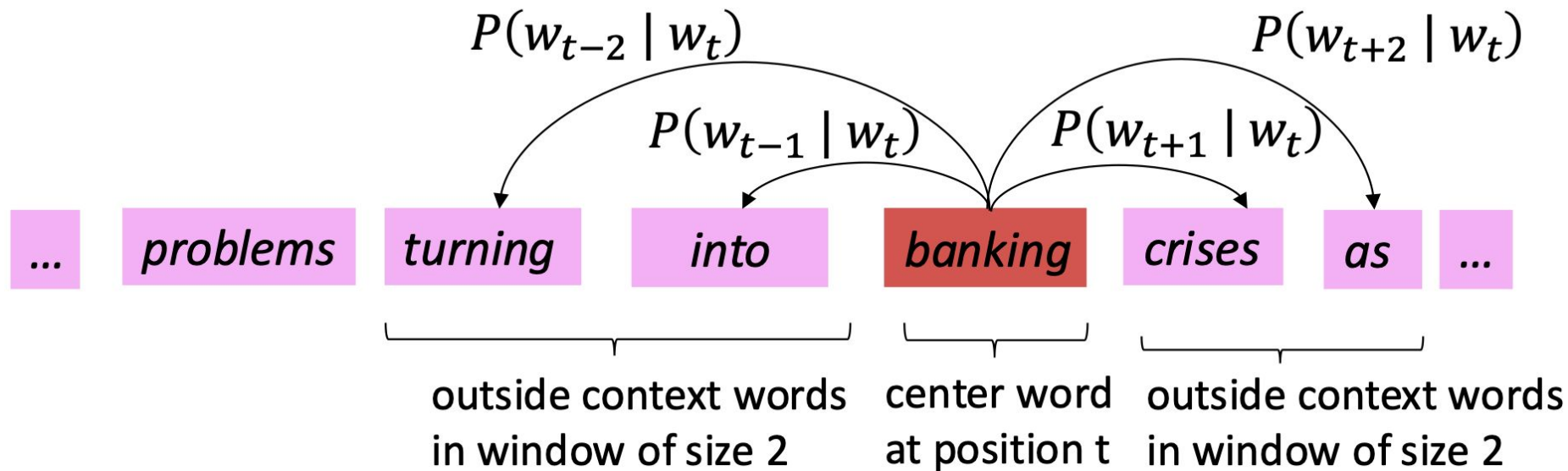


Figure 1: New model architectures. The CBOW architecture predicts the current word based on the context, and the Skip-gram predicts surrounding words given the current word.

Word2Vec: overview



Word2Vec: overview



Word2Vec: objective

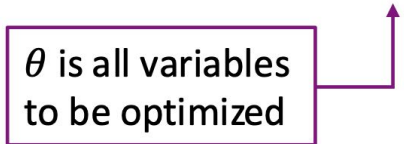
For each position $t = 1, \dots, T$, predict context words within a window of fixed size m , given center word w_t .

Word2Vec: objective

For each position $t = 1, \dots, T$, predict context words within a window of fixed size m , given center word w_t .

Likelihood = $L(\theta) = \prod_{t=1}^T \prod_{\substack{-m \leq j \leq m \\ j \neq 0}} P(w_{t+j} \mid w_t; \theta)$

θ is all variables to be optimized



Word2Vec: objective

For each position $t = 1, \dots, T$, predict context words within a window of fixed size m , given center word w_t .

Likelihood = $L(\theta) = \prod_{t=1}^T \prod_{\substack{-m \leq j \leq m \\ j \neq 0}} P(w_{t+j} | w_t; \theta)$

θ is all variables to be optimized

Q: How to compute $P(o|c)$?

The loss is negative log likelihood:

$$J(\theta) = -\frac{1}{T} \log L(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{\substack{-m \leq j \leq m \\ j \neq 0}} \log P(w_{t+j} | w_t; \theta)$$

Minimizing objective function $\Leftrightarrow$ Maximizing predictive accuracy

Word2Vec: objective

For each position $t = 1, \dots, T$, predict context words within a window of fixed size m , given center word w_t .

Likelihood = $L(\theta) = \prod_{t=1}^T \prod_{\substack{-m \leq j \leq m \\ j \neq 0}} P(w_{t+j} | w_t; \theta)$

θ is all variables to be optimized

The loss is negative log likelihood:

$$P(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)}$$

$$J(\theta) = -\frac{1}{T} \log L(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{\substack{-m \leq j \leq m \\ j \neq 0}} \log P(w_{t+j} | w_t; \theta)$$

Minimizing objective function $\Leftrightarrow$ Maximizing predictive accuracy

Window-based co-occurrence matrix

Window length 1 (more common: 5–10)

Example corpus: [I like deep learning, I like NLP, I enjoy flying]

counts	I	like	enjoy	deep	learning	NLP	flying	.
I	0	2	1	0	0	0	0	0
like	2	0	0	1	0	1	0	0
enjoy	1	0	0	0	0	0	1	0
deep	0	1	0	0	1	0	0	0
learning	0	0	0	1	0	0	0	1
NLP	0	1	0	0	0	0	0	1
flying	0	0	1	0	0	0	0	1
.	0	0	0	0	1	1	1	0

Co-occurrence vectors

1. Vectors increase in size with vocabulary
2. Very high dimensional: require a lot of storage
3. **Sparse**

Q: How to fix this?

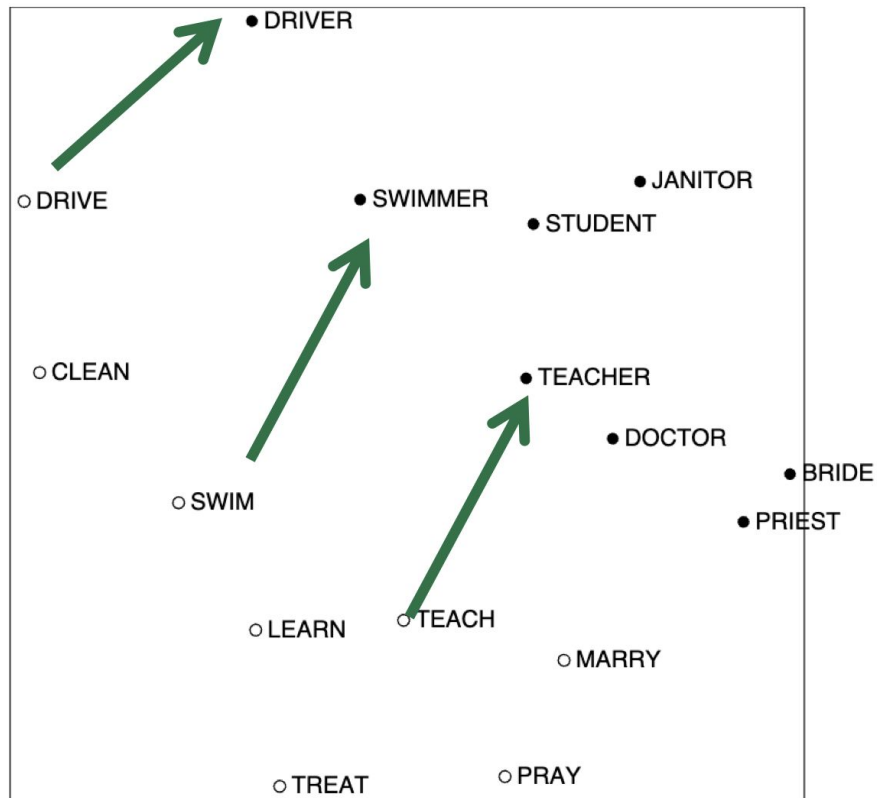
Co-occurrence vectors

1. Vectors increase in size with vocabulary
2. Very high dimensional: require a lot of storage
3. **Sparse**

Solution:

1. Use dimensionality reduction methods to produce low-dimensional vectors (e.g. SVD/PCA)
2. I.e. store “most” of the important information in a fixed, small number of dimensions: a dense vector
3. Usually 25–1000 dimensions

Semantic patterns



GloVe [EMNLP 2014]

GloVe captures **ratios of co-occurrence probabilities** as linear meaning components in a word vector space

$P(i | j)$ is probability that word i appear in the context of word j

Table 1: Co-occurrence probabilities for target words *ice* and *steam* with selected context words from a 6 billion token corpus. Only in the ratio does noise from non-discriminative words like *water* and *fashion* cancel out, so that large values (much greater than 1) correlate well with properties specific to ice, and small values (much less than 1) correlate well with properties specific of steam.

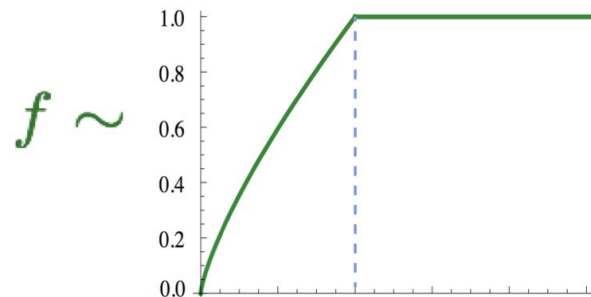
Probability and Ratio	$k = \textit{solid}$	$k = \textit{gas}$	$k = \textit{water}$	$k = \textit{fashion}$
$P(k \textit{ice})$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k \textit{steam})$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$P(k \textit{ice})/P(k \textit{steam})$	8.9	8.5×10^{-2}	1.36	0.96

GloVe

$$w_i \cdot w_j = \log P(i|j)$$

$$w_x \cdot (w_a - w_b) = \log \frac{P(x|a)}{P(x|b)}$$

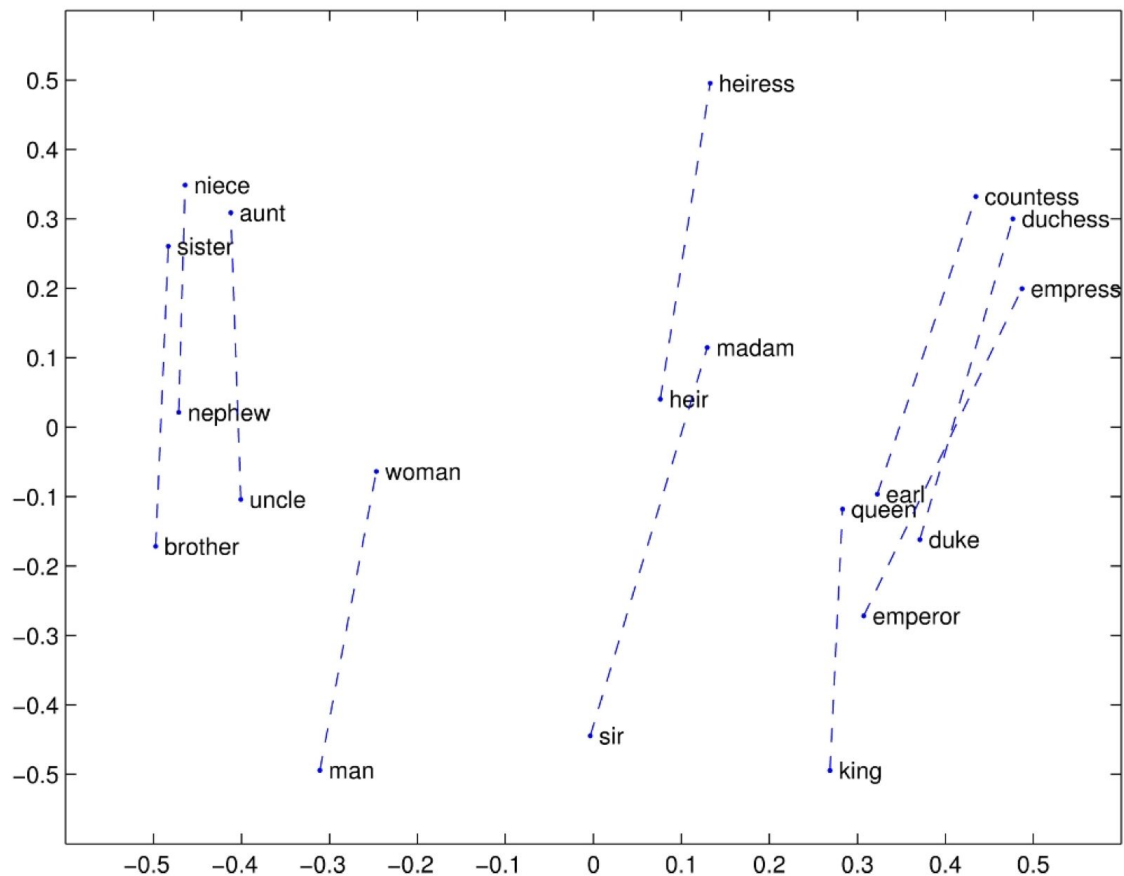
$$J = \sum_{i,j=1}^V f(X_{ij}) \left(w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2$$



Fast training

Scalable to huge corpora

GloVe: visualization



Language modeling

Language Modeling is the task of predicting what word comes next

the students opened their _____

Language modeling

Language Modeling is the task of predicting what word comes next

the students opened their {books: .4, laptops: .4, minds: .19,...}

Q: How to formalise?

Language modeling

Language Modeling is the task of predicting what word comes next

the students opened their {books: .4, laptops: .4, minds: .19,...}

More formally: given a sequence of words $x^{(1)}, x^{(2)}, \dots, x^{(t)}$,
compute the probability distribution of the next word $x^{(t+1)}$:

$$P(x^{(t+1)} \mid x^{(t)}, \dots, x^{(1)})$$

where $x^{(t+1)}$ can be any word in the vocabulary $V = \{w_1, \dots, w_{|V|}\}$

A system that does this is called a Language Model

Language modeling

You can also think of a Language Model as a system that assigns a probability to a piece of text

$$\begin{aligned} P(\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(T)}) &= P(\mathbf{x}^{(1)}) \times P(\mathbf{x}^{(2)} | \mathbf{x}^{(1)}) \times \dots \times P(\mathbf{x}^{(T)} | \mathbf{x}^{(T-1)}, \dots, \mathbf{x}^{(1)}) \\ &= \prod_{t=1}^T P(\mathbf{x}^{(t)} | \mathbf{x}^{(t-1)}, \dots, \mathbf{x}^{(1)}) \end{aligned}$$


This is what our LM provides

Language models



🔍 Ukrainian Q



ukrainian catholic university

Український Католицький Університет — Університет у Львові, Україна



ukrainian currency



ukrainian currency to usd



ukrainian cuisine

Language models

Що у вас сьогодні на думці?

+ Запитайте будь-що



Q: Why should we care about Language Modeling?

Why should we care about Language Modeling:

1. It is a **benchmark task** that helps us **measure our progress** on predicting language use
2. It is a **subcomponent** of many NLP tasks, especially those involving **generating text** or **estimating the probability of text**:
 - a. Predictive typing
 - b. Speech recognition
 - c. Handwriting recognition
 - d. Spelling/grammar correction
 - e. Authorship identification
 - f. Machine translation
 - g. Summarization
 - h. Dialogue
 - i. etc.

A fixed-window neural Language Model

output distribution

$$\hat{y} = \text{softmax}(Uh + b_2) \in \mathbb{R}^{|V|}$$

hidden layer

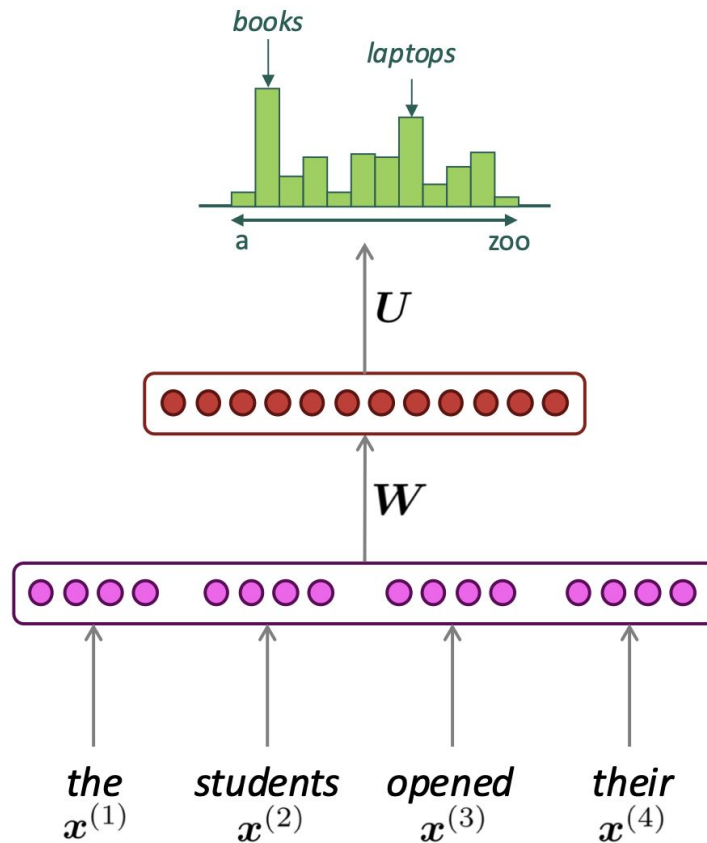
$$h = f(We + b_1)$$

concatenated word embeddings

$$e = [e^{(1)}; e^{(2)}; e^{(3)}; e^{(4)}]$$

words / one-hot vectors

$$x^{(1)}, x^{(2)}, x^{(3)}, x^{(4)}$$



A fixed-window neural Language Model

Q: Problems?

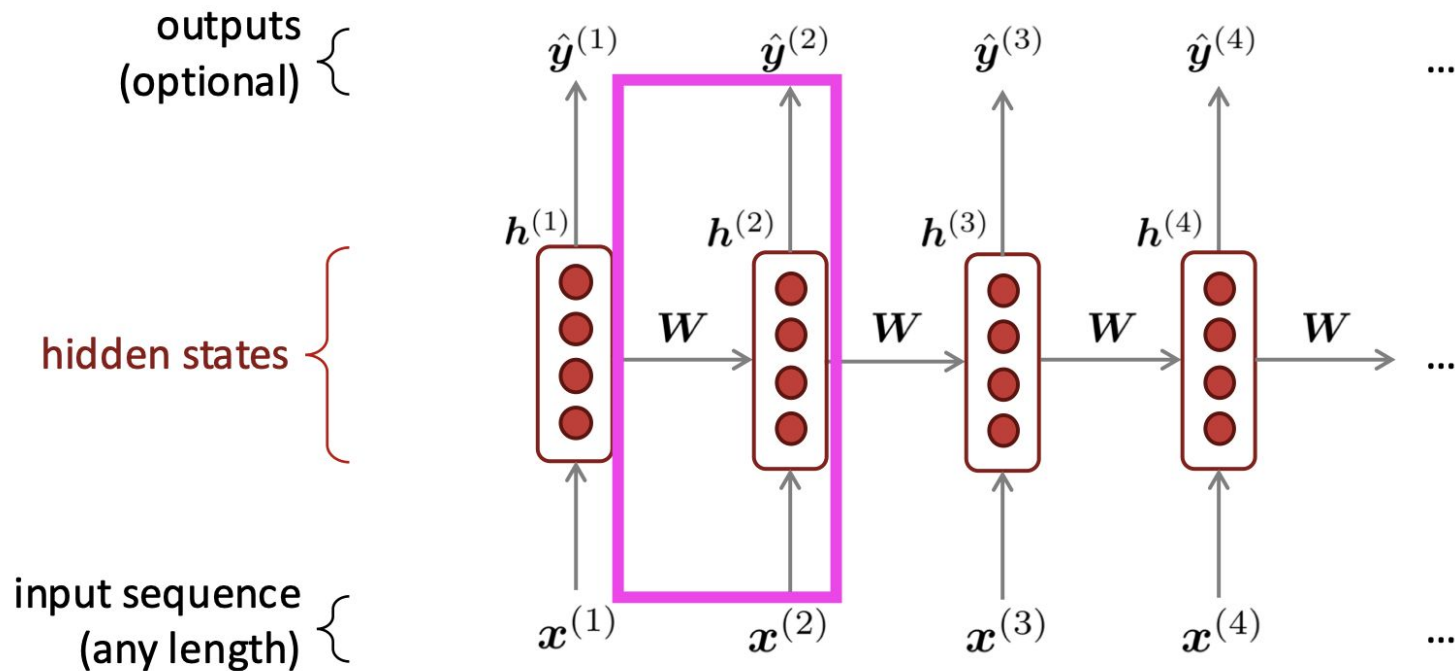
A fixed-window neural Language Model

Problems:

1. Fixed window is **too small**
2. Enlarging window enlarges W
Window can never be large enough!
3. $x(1)$ and $x(2)$ are multiplied by completely different weights in W .
No symmetry in how the inputs are processed.
4. We need a neural architecture that can process any length input

Recurrent Neural Networks (RNN)

Core idea: Apply the same weights W repeatedly



output distribution

$$\hat{\mathbf{y}}^{(t)} = \text{softmax}(\mathbf{U}\mathbf{h}^{(t)} + \mathbf{b}_2) \in \mathbb{R}^{|V|}$$

hidden states

$$\mathbf{h}^{(t)} = \sigma(\mathbf{W}_h \mathbf{h}^{(t-1)} + \mathbf{W}_e \mathbf{e}^{(t)} + \mathbf{b}_1)$$

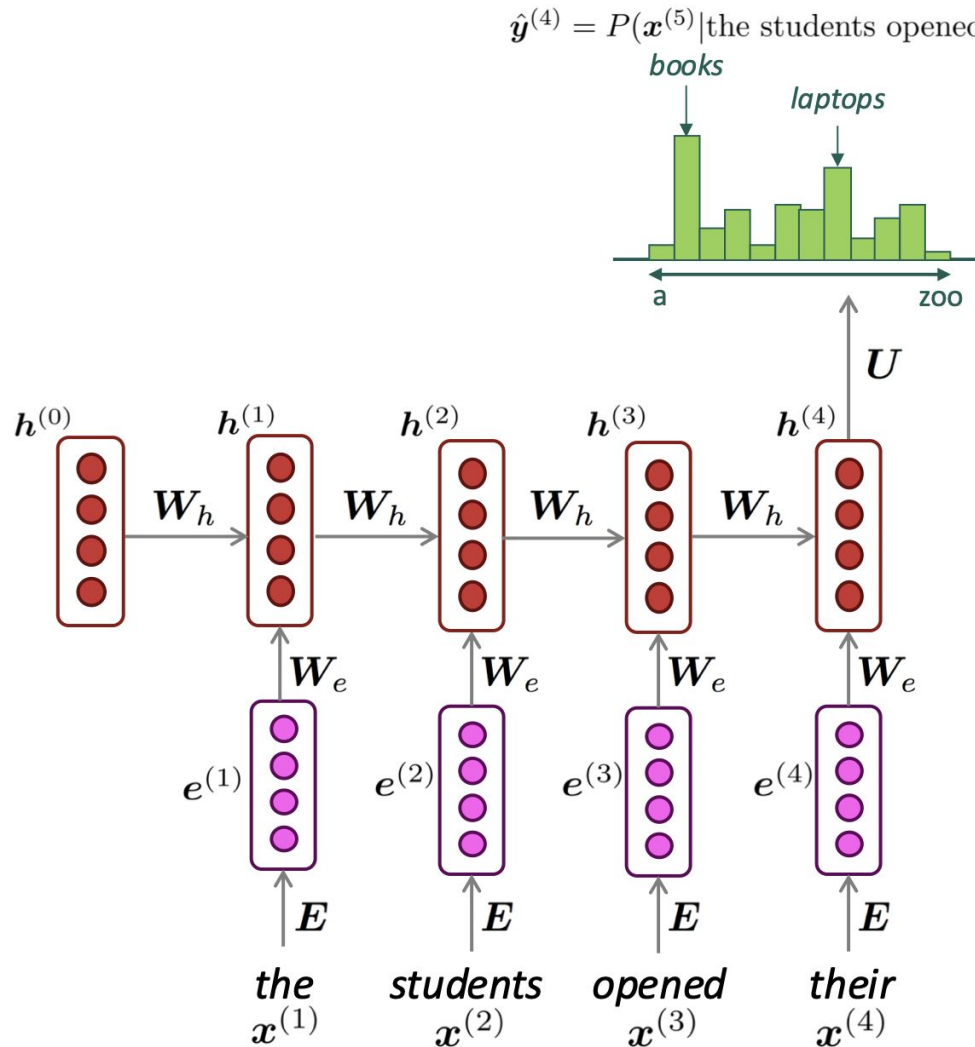
$\mathbf{h}^{(0)}$ is the initial hidden state

word embeddings

$$\mathbf{e}^{(t)} = \mathbf{E}\mathbf{x}^{(t)}$$

words / one-hot vectors

$$\mathbf{x}^{(t)} \in \mathbb{R}^{|V|}$$



RNN-LM

Advantages:

- Can process any length input
- Computation for step t can (in theory) use information from many steps back
- Model size doesn't increase for longer input context
- Same weights applied on every timestep, so there is symmetry in how inputs are processed.

Disadvantages:

- Recurrent computation is slow
- In practice, difficult to access information from many steps back

Training an RNN-LM

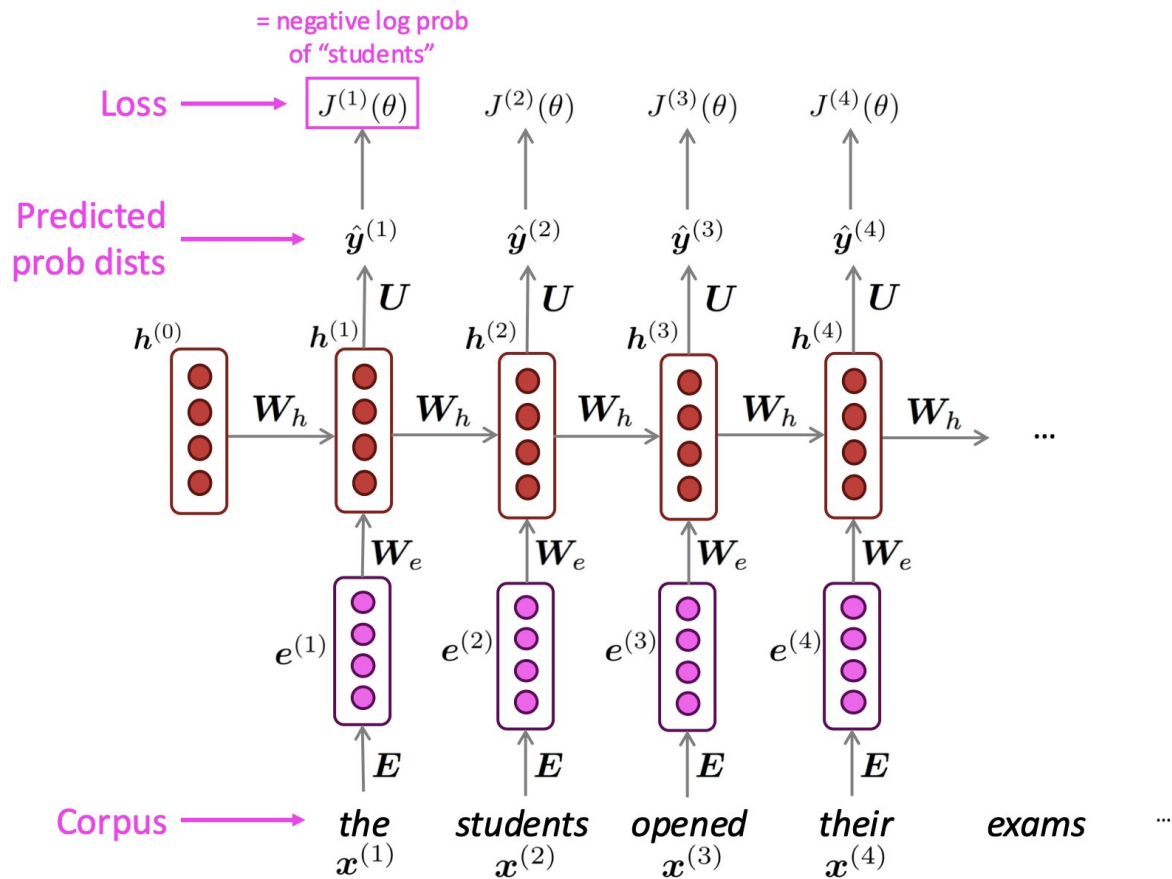
1. Get a big corpus of text which is a sequence of words
2. Feed into RNN-LM; compute output distribution; i.e., **predict probability dist of every word**, given words so far
3. Loss function on step t is **cross-entropy** between predicted probability distribution and the true next word

$$J^{(t)}(\theta) = CE(\mathbf{y}^{(t)}, \hat{\mathbf{y}}^{(t)}) = - \sum_{w \in V} \mathbf{y}_w^{(t)} \log \hat{\mathbf{y}}_w^{(t)} = - \log \hat{\mathbf{y}}_{\mathbf{x}_{t+1}}^{(t)}$$

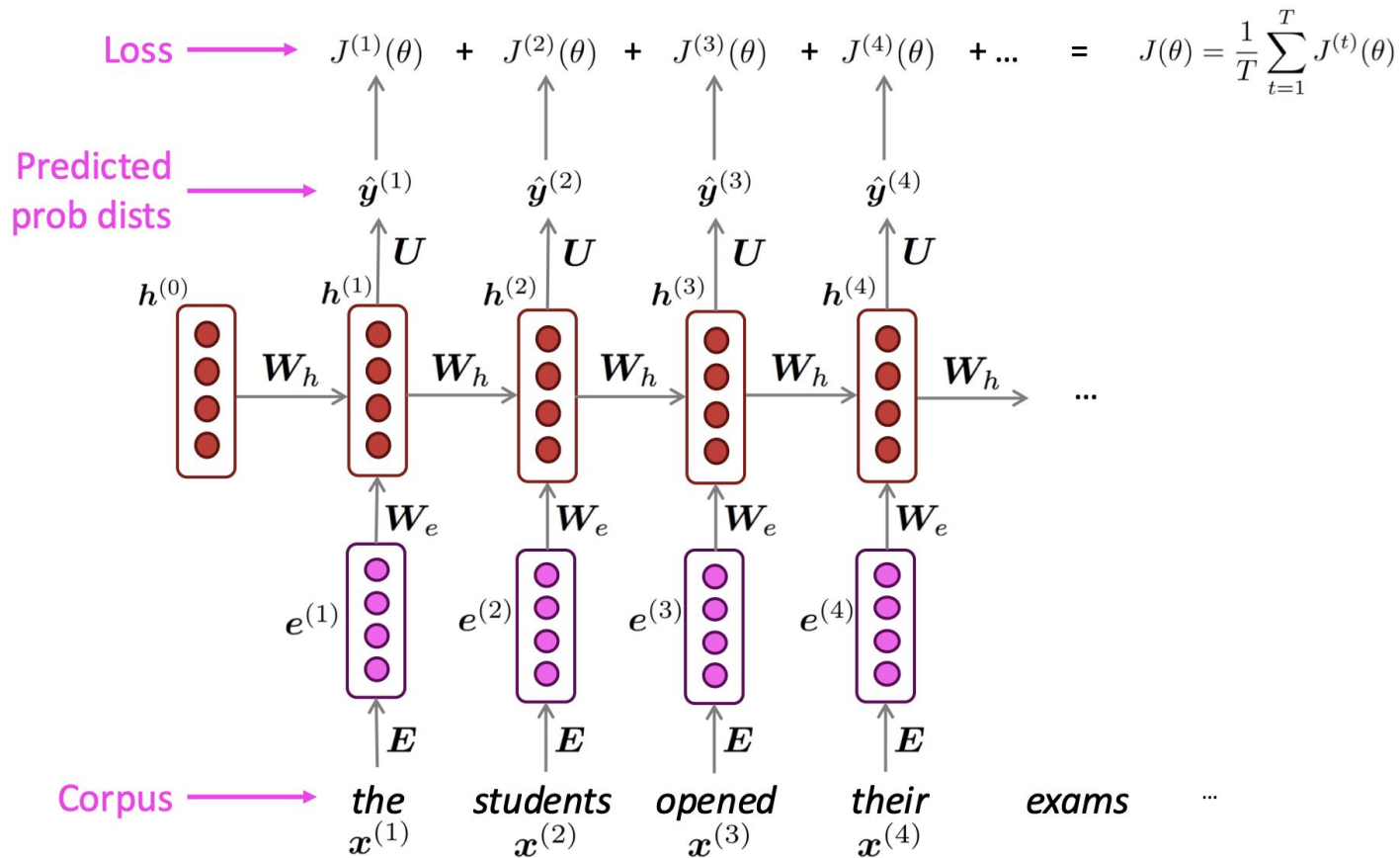
4. **Average** this to get overall loss for entire training set:

$$J(\theta) = \frac{1}{T} \sum_{t=1}^T J^{(t)}(\theta) = \frac{1}{T} \sum_{t=1}^T - \log \hat{\mathbf{y}}_{\mathbf{x}_{t+1}}^{(t)}$$

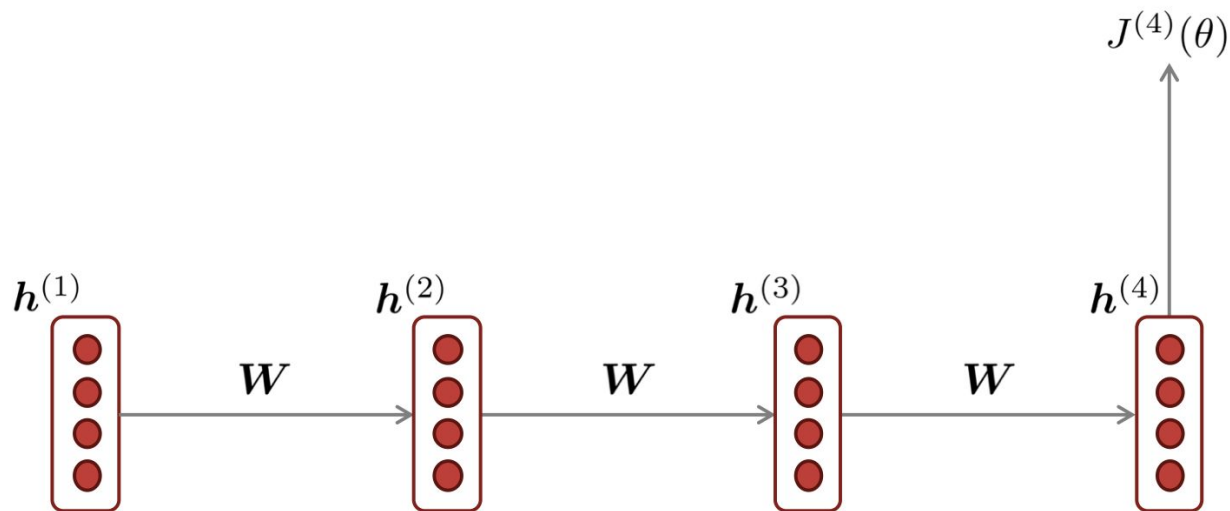
Training an RNN-LM



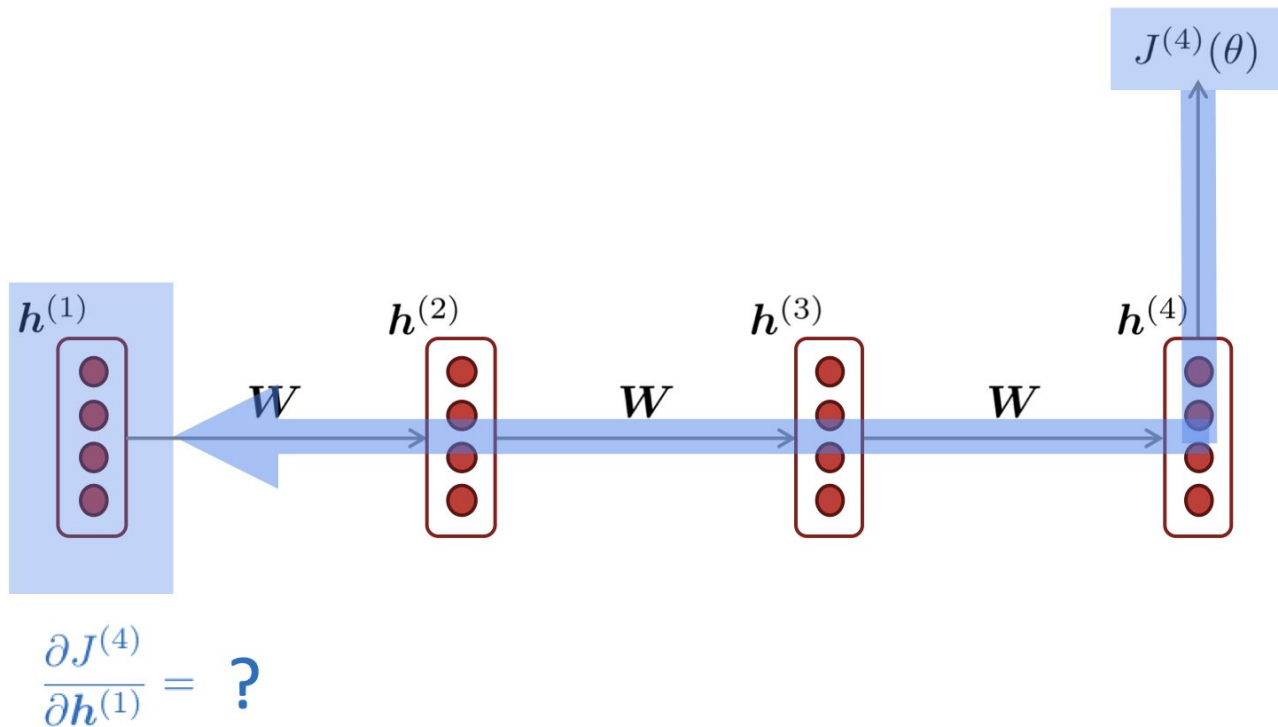
Training an RNN-LM



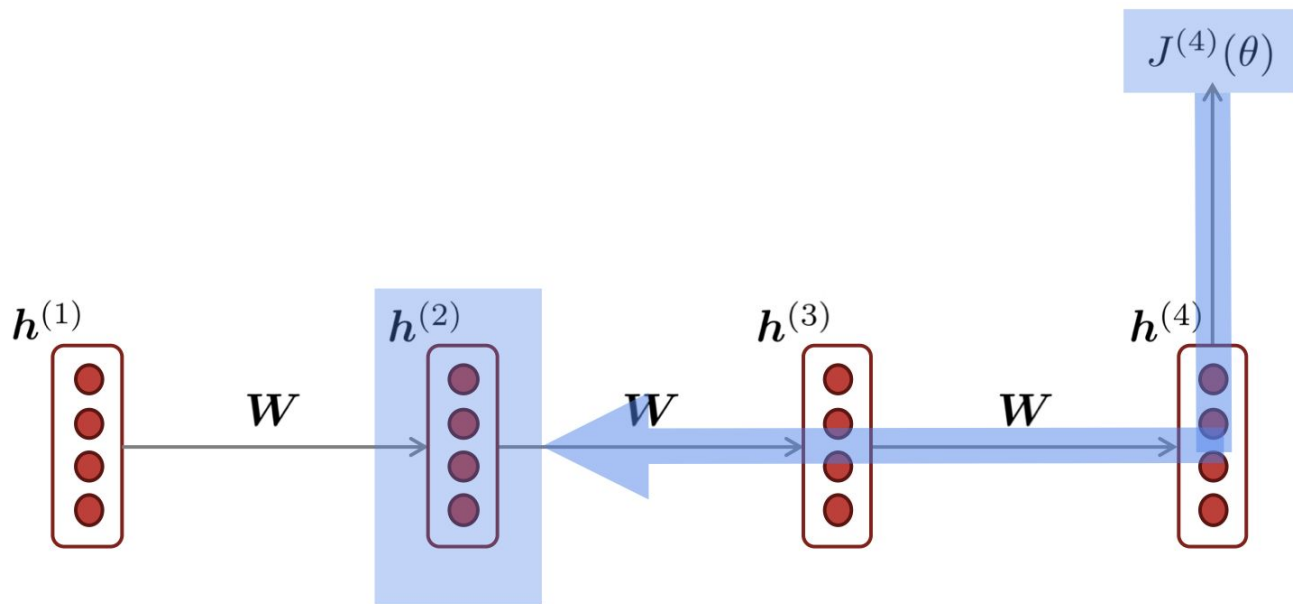
Problems with RNNs: Vanishing and Exploding Gradients



Vanishing Gradients



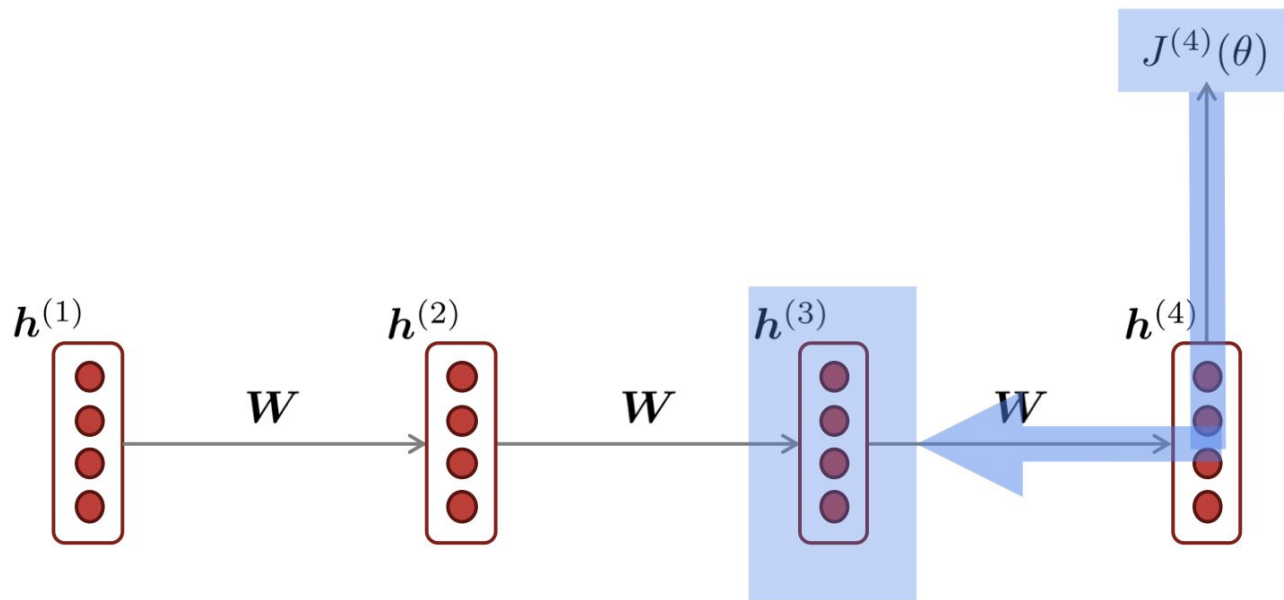
Vanishing Gradients



$$\frac{\partial J^{(4)}}{\partial h^{(1)}} = \frac{\partial h^{(2)}}{\partial h^{(1)}} \times \frac{\partial J^{(4)}}{\partial h^{(2)}}$$

chain rule!

Vanishing Gradients

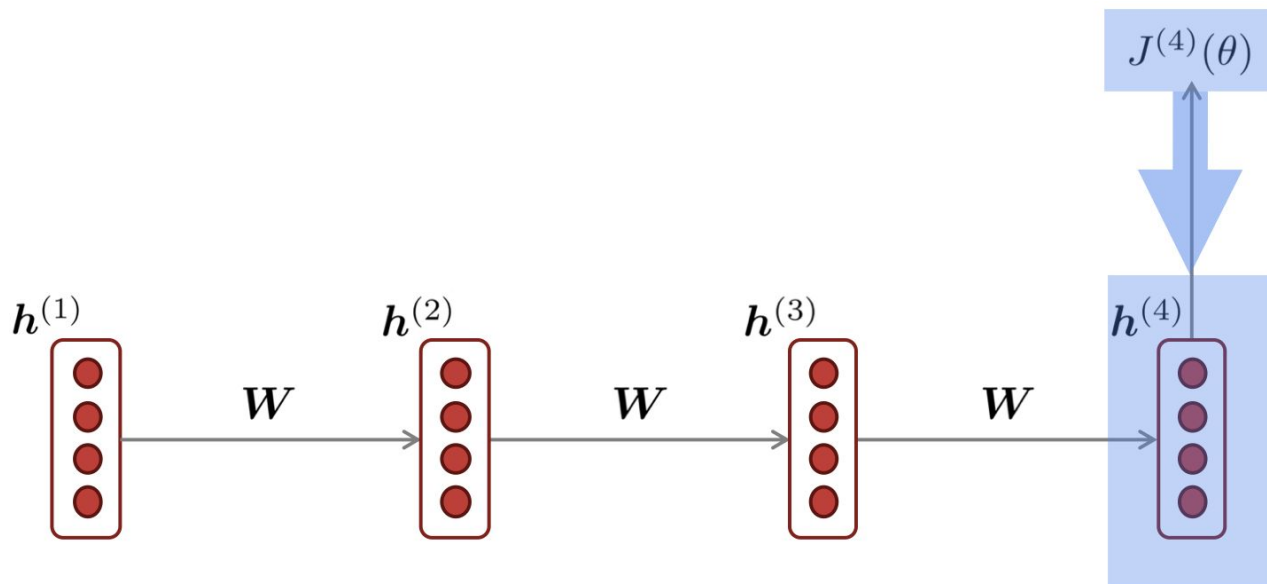


$$\frac{\partial J^{(4)}}{\partial h^{(1)}} = \frac{\partial h^{(2)}}{\partial h^{(1)}} \times$$

$$\frac{\partial h^{(3)}}{\partial h^{(2)}} \times \frac{\partial J^{(4)}}{\partial h^{(3)}}$$

chain rule!

Vanishing Gradients



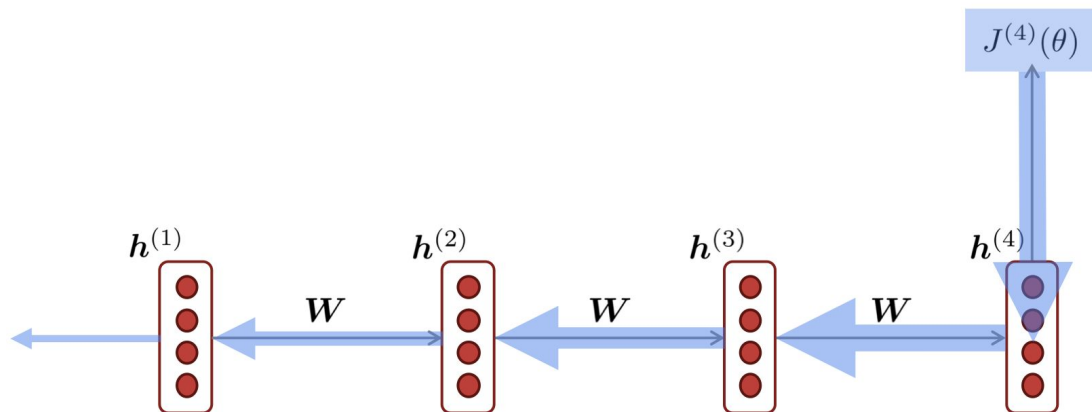
$$\frac{\partial J^{(4)}}{\partial h^{(1)}} = \frac{\partial h^{(2)}}{\partial h^{(1)}} \times$$

$$\frac{\partial h^{(3)}}{\partial h^{(2)}} \times$$

$$\frac{\partial h^{(4)}}{\partial h^{(3)}} \times \frac{\partial J^{(4)}}{\partial h^{(4)}}$$

chain rule!

Vanishing Gradients

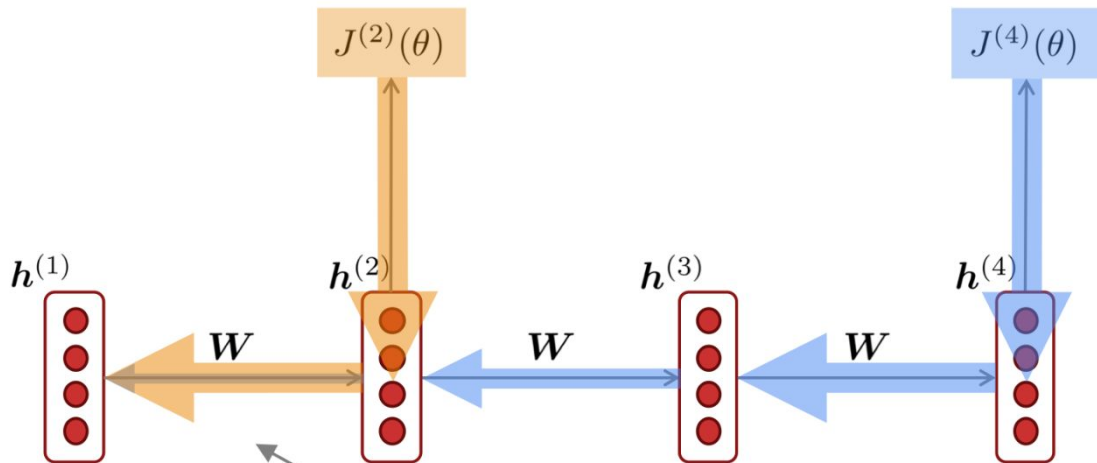


$$\frac{\partial J^{(4)}}{\partial \mathbf{h}^{(1)}} = \frac{\partial \mathbf{h}^{(2)}}{\partial \mathbf{h}^{(1)}} \times \frac{\partial \mathbf{h}^{(3)}}{\partial \mathbf{h}^{(2)}} \times \frac{\partial \mathbf{h}^{(4)}}{\partial \mathbf{h}^{(3)}} \times \frac{\partial J^{(4)}}{\partial \mathbf{h}^{(4)}}$$

What happens if these are small?

Vanishing gradient problem:
When these are small, the gradient signal gets smaller and smaller as it backpropagates further

Why is vanishing gradient a problem?



Gradient signal from far away is lost because it's much smaller than gradient signal from close-by.

So, model weights are updated only with respect to near effects, not long-term effects.

Effect of vanishing gradient on RNN-LM

LM task: *When she tried to print her tickets, she found that the printer was out of toner. She went to the stationery store to buy more toner. It was very overpriced. After installing the toner into the printer, she finally printed her _____*

To learn from this training example, the RNN-LM needs to **model the dependency** between “tickets” on the 7th step and the target word “tickets” at the end.

But if the gradient is small, the model **can't learn this dependency**

So, the model is **unable to predict similar long-distance dependencies** at test time

Why is exploding gradient a problem?

If the gradient becomes too big, then the SGD update step becomes too big:

$$\theta^{new} = \theta^{old} - \overset{\text{learning rate}}{\alpha} \underbrace{\nabla_{\theta} J(\theta)}_{\text{gradient}}$$

This can cause **bad updates**: we take too large a step and reach a weird and bad parameter configuration (with large loss)

In the worst case, this will result in **Inf** or **NaN** in your network (then you have to restart training from an earlier checkpoint)

Q: How to fix exploding gradient problem?

How to fix exploding gradient problem:

Gradient clipping: if the norm of the gradient is greater than some threshold, scale it down before applying SGD update

Algorithm 1 Pseudo-code for norm clipping

```
 $\hat{\mathbf{g}} \leftarrow \frac{\partial \mathcal{E}}{\partial \theta}$   
if  $\|\hat{\mathbf{g}}\| \geq threshold$  then  
     $\hat{\mathbf{g}} \leftarrow \frac{threshold}{\|\hat{\mathbf{g}}\|} \hat{\mathbf{g}}$   
end if
```

Q: How to fix vanishing gradient problem?

How to fix vanishing gradient problem:

The main problem is that it's too difficult for the RNN to learn to preserve information over many timesteps.

In a vanilla RNN, the hidden state is constantly being **rewritten**

How about an RNN with separate **memory** which is added to?

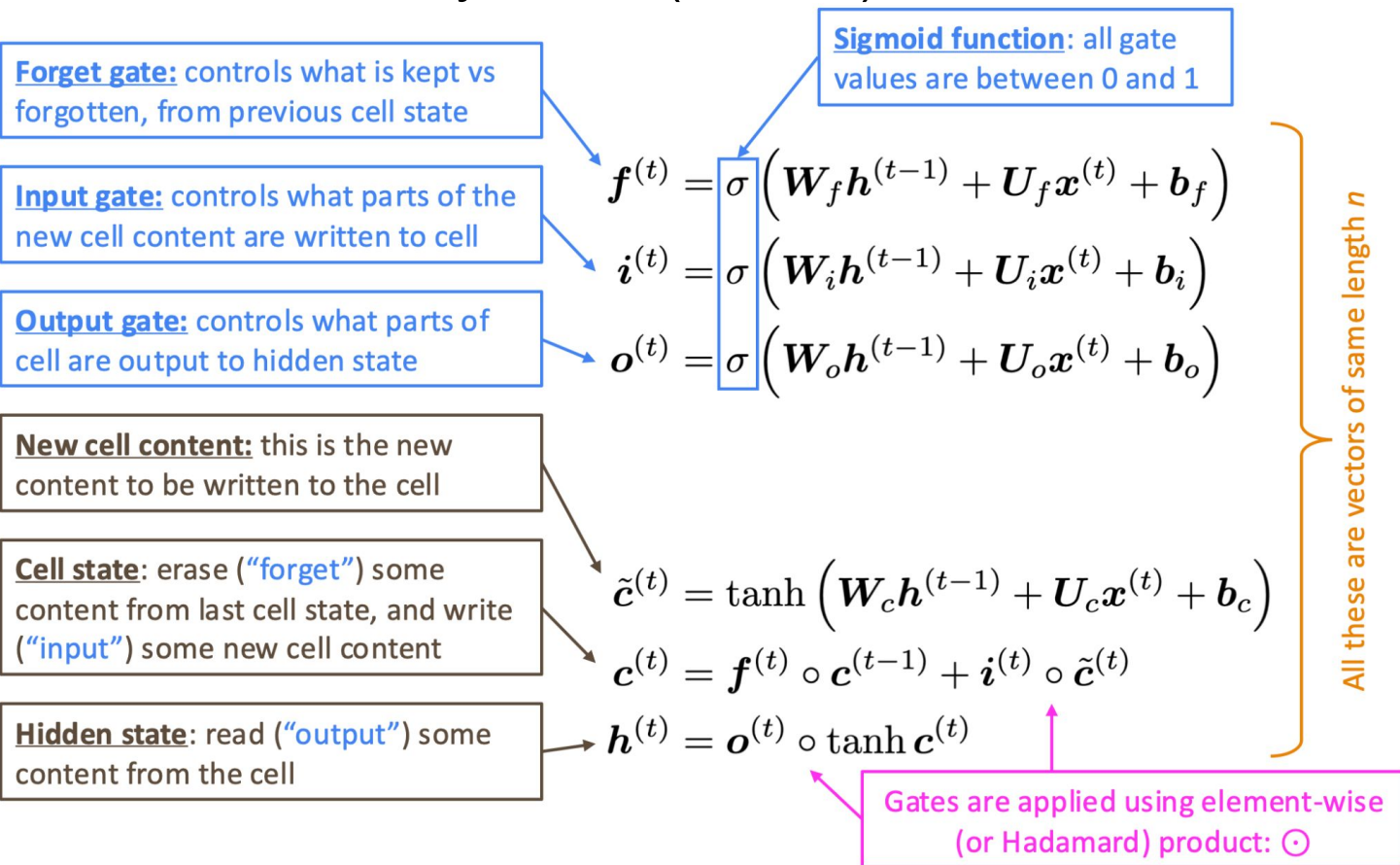
Long Short-Term Memory RNNs (LSTMs)

- A type of RNN proposed by Hochreiter and Schmidhuber in 1997 as a solution to the problem of vanishing gradients
- But only really became well-known after Hinton brought it to Google in 2013

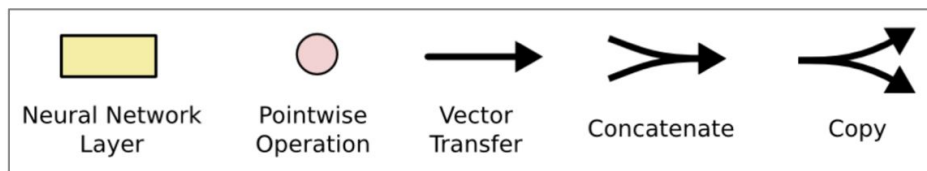
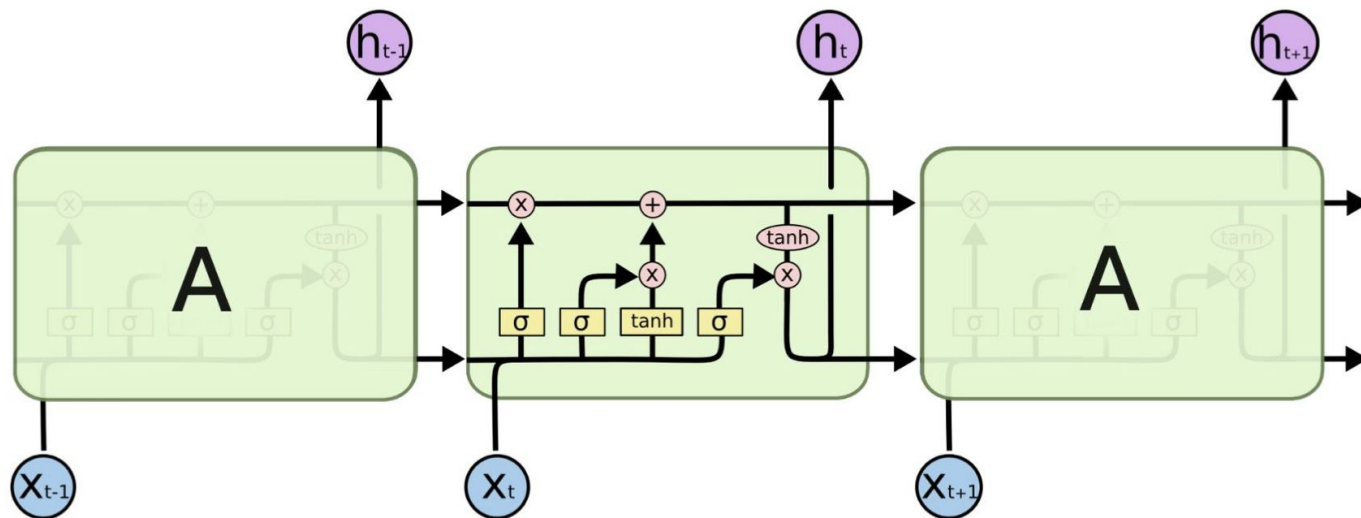
Long Short-Term Memory RNNs (LSTMs)

- On step t , there is a **hidden state** $h(t)$ and a **cell state** $c(t)$ (both are vectors)
- The cell stores **long-term information**
- The LSTM can **read**, **erase**, and **write** information from the cell
 - The cell becomes conceptually rather like RAM in a computer
- The selection of which information is erased/written/read is controlled by three corresponding **gates**
- The gates are also vectors of length n
- On each timestep, each element of the gates can be **open** (1), **closed** (0), or somewhere in-between
- The gates are **dynamic**: their value is computed based on the current context

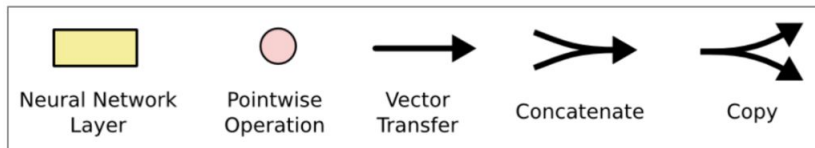
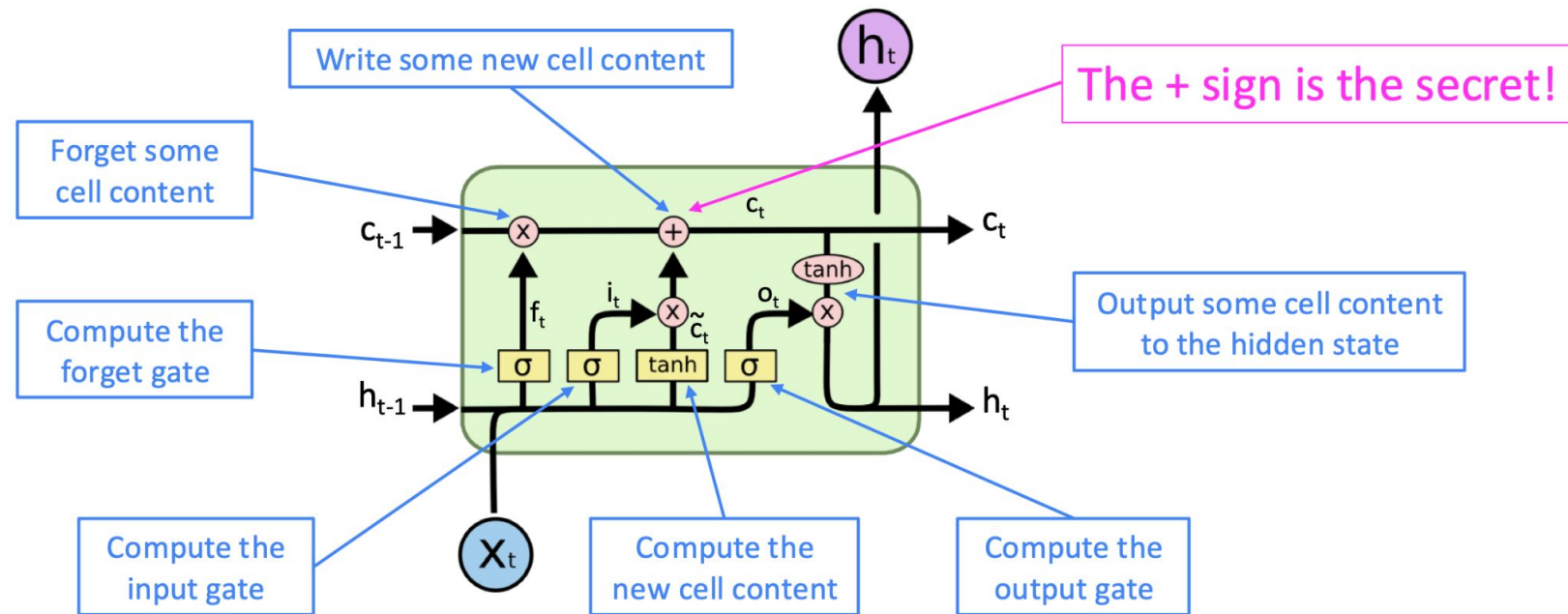
Long Short-Term Memory RNNs (LSTMs)



Long Short-Term Memory RNNs (LSTMs)



Long Short-Term Memory RNNs (LSTMs)



How does LSTM solve vanishing gradients? Does it?

- The LSTM architecture makes it **much easier** for an RNN to **preserve information over many timesteps**
- It's harder for a vanilla RNN to learn a recurrent weight matrix W_h that preserves info in the hidden state
- In practice, you get about **100 timesteps rather than about 7**

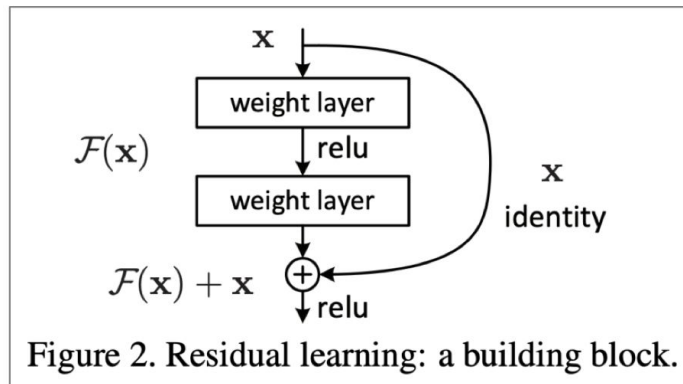
However, there are **alternative ways** of creating more direct and linear pass-through connections in models for long distance dependencies

Q: What are these?

How does LSTM solve vanishing gradients? Does it?

- The LSTM architecture makes it **much easier** for an RNN to **preserve information over many timesteps**
- It's harder for a vanilla RNN to learn a recurrent weight matrix W_h that preserves info in the hidden state
- In practice, you get about **100 timesteps rather than about 7**

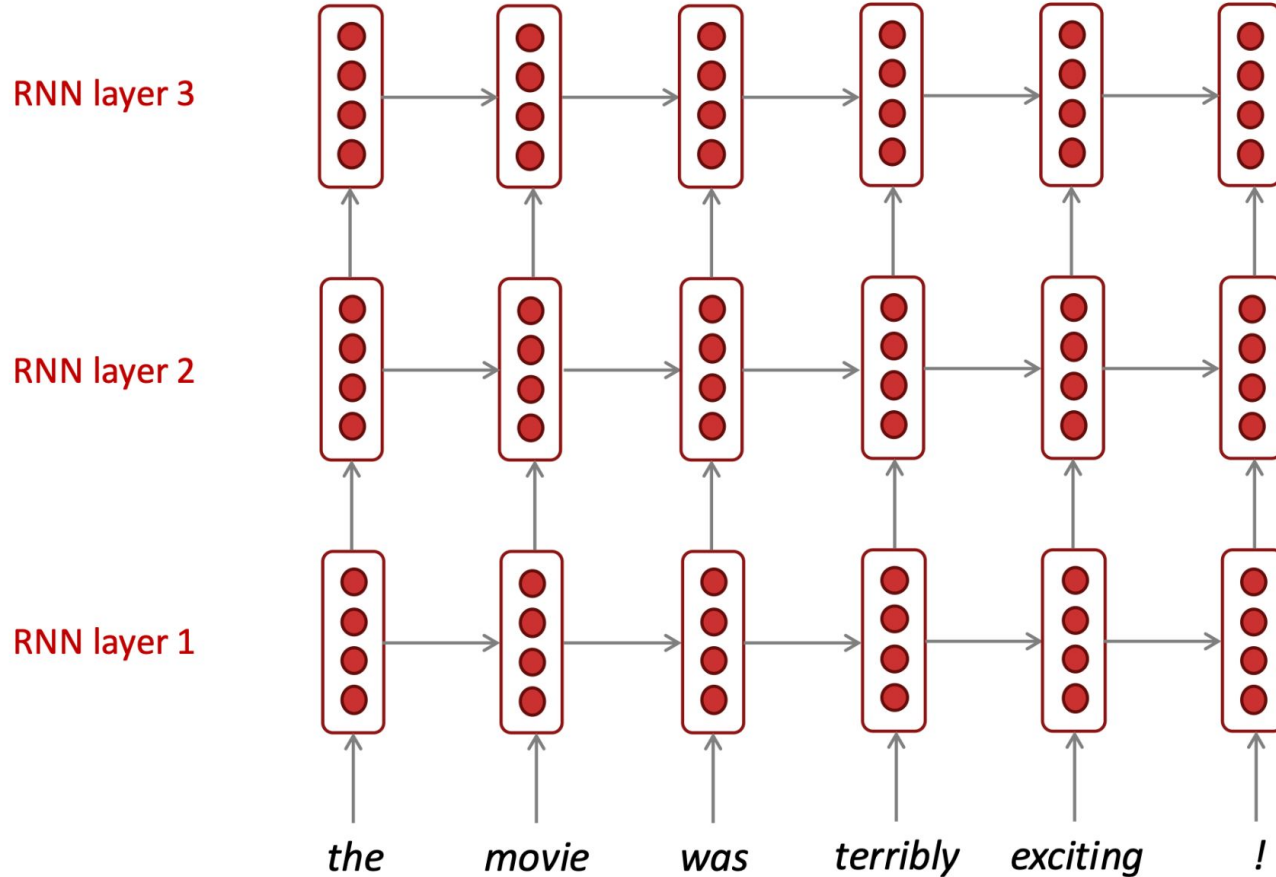
However, there are **alternative ways** of creating more direct and linear pass-through connections in models for long distance dependencies



Multi-layer RNNs

What if we stack them?

The hidden states from RNN layer i
are the inputs to RNN layer $i+1$



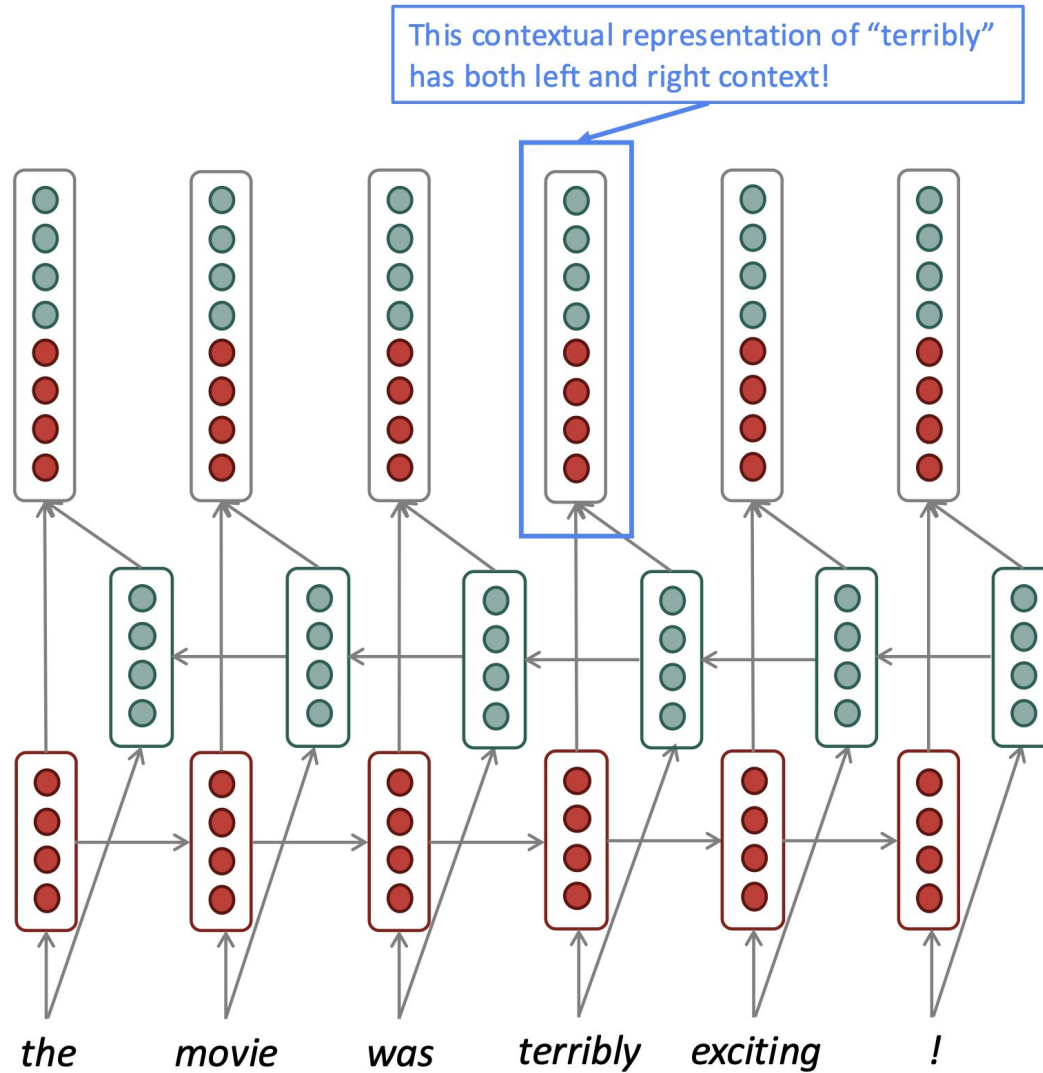
Bidirectional RNNs

What if we run in both directions?

Concatenated
hidden states

Backward RNN

Forward RNN



Q&A